

FPGA Design of a Fast 32-bit Floating Point Multiplier Unit

Anna Jain, Baisakhy Dash, Ajit Kumar Panda, *Member, IEEE*, Muchharla Suresh, *Member, IEEE*

Abstract- An architecture for a fast 32-bit floating point multiplier compliant with the single precision IEEE 754-2008 standard has been proposed in this paper. This design intends to make the multiplier faster by reducing the delay caused by the propagation of the carry by implementing adders having the least power delay constant. The implementation of the multiplier module has been done in a top down approach. The sub-modules have been written in Verilog HDL and then synthesized and simulated using the Xilinx ISE 12.1 targeted on the Spartan 3E FPGA.

I. INTRODUCTION

With the advent of technology, the demand of high-speed digital systems is on the rise and the multiplier is a ubiquitous unit in almost every digital system. Compared to other operations in an arithmetic logic unit the multiplier consumes more time and power. Hence researchers have always been trying to design multipliers which incorporate an optimal combination in terms speed, power and area.

In computing, floating point describes a method of representing real numbers in a way that can support a wide range of values. Floating point units are widely used in a dynamic range of engineering and technology applications. This demands for the development of faster floating point arithmetic circuits.

In this paper we propose an architecture for a fast floating point multiplier compliant with the single precision IEEE 754-2008 standard. The major issue in the implementation of high speed multiplier circuit is the delay due to the propagation of carry in every component used in its design. In this proposed architecture we are trying to minimize the carry propagation at every level possible.

Modern Field Programmable Gate Arrays (FPGAs) are a suitable solution that provides thousands of logic elements and dedicated blocks as well as several desired properties such as intrinsic parallelism, flexibility, low cost and customizable approaches. All this allows for a better performance and accelerated execution of the involved algorithms. FPGAs are quickly becoming suitable for major floating point computations.

To attain a generic design, Verilog hardware description language was used for design entry of the entire multiplier unit as it presents a tremendous productivity improvement for circuit designers and descriptions of large circuits can be written in a relatively compact and concise form.

Over the years, several different floating-point representations have been used in computers; however, for the last ten years the most commonly encountered representation is that defined by the IEEE Standard for Floating-Point Arithmetic (IEEE 754) [1]. It is a technical

standard established by the Institute of Electrical and Electronics Engineers (IEEE) and the most widely used standard for floating-point computation, followed by many hardware and software implementations. Single precision representation occupies 32 bits: a sign bit, 8 bits for exponent and 23 for the mantissa. It also specifies standards for arithmetic operations and rounding algorithms.

The rest of the paper is organized as follows. Section 2 presents the proposed floating point multiplier design and explains the architectural details. Section 3 lists the progress and proposals to achieve the objective of the paper. The implementation is described in section 4 and with section 5 we conclude the paper.

II. FLOATING POINT MULTIPLIER DESIGN

The floating point multiplication is carried out in three parts [2]:

In the first part, we determine the sign of the product by performing a xor operation on the sign bits of the two operands.

In the second part, the exponent bits of the operands are passed to an adder stage and a bias of 127 is subtracted from the obtained output. The addition and bias subtraction operations are both implemented using 8-bit kogge-stone adders. Overflow and underflow conditions are indicated by setting the respective flags.

In the third and most important stage, we find the product of the mantissa bits. The multiplication of mantissa bits is performed in the following stages.

A. Partial product generator: There are various ways of generating partial products for a given multiplier [3]. The ones that we have considered are booth encoding and radix-4 booth encoding. The radix-4 booth encoding was found to be faster so it has been implemented in the final multiplier architecture. The output of this stage is twelve partial products.

B. Partial product accumulator: The 24-bit partial products obtained from the previous stage are shifted appropriately in a shifter module and then accumulated using multi-operand tree adders like Wallace tree, dadda tree, overturned stairs tree and 4:2 compressor tree. In our design we have used the Wallace tree structure which comprises of carry-save adders. Use of carry-save adders greatly reduces the carry propagation time of this stage.

C. Final stage adder: The 48-bit sum and carry outputs obtained from the partial product accumulator are added in the final stage adder to give the product of the mantissas. This stage calls for the implementation of adders with less delay and greater speed. Studying and comparing the power and delay

characteristics of various adders, we concluded that the Kogge-Stone adder is the fastest.

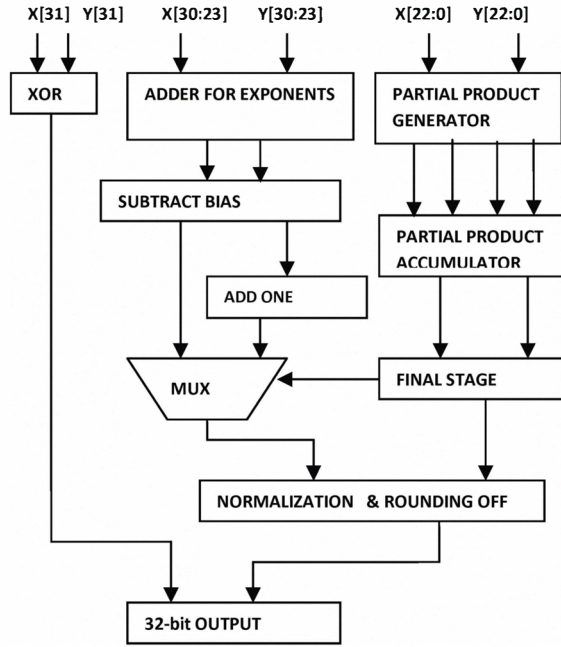


Fig.1. Multiplier module

D. Normalization and rounding: In this stage, the product of the mantissas is normalised and truncated. To do so, the leading-one is detected and the exponent is adjusted accordingly. The leading one is the implied bit and hence dropped. The remaining bits are truncated to a 26-bit value. A few extra bits from the truncated value are used for accuracy and extra precision namely the guard, round and sticky bits [4]. The truncated value is finally rounded off using the rounding to nearest even technique to give the 23 bit mantissa of the product.

To avoid unnecessary calculations in the event of occurrence of zero in the input, a zero detect block is included in the multiplier architecture.

III. IMPLEMENTATION

The main objective of this paper is to increase the multiplier speed by minimizing the overall delay. As is obvious from our proposed architecture, almost every module is built on the fundamental unit of an adder. So, we surveyed the different fast adders available. Studying and comparing the power and delay characteristics of the adders, we concluded that the Kogge-Stone adder is the fastest and then proceeded to implement the same at every stage.

The various sub modules of the single precision floating point multiplier have been individually designed in verilog HDL, synthesized and simulated using the Xilinx ISE 12.1 targeted on the Spartan 3E FPGA. The blocks have then been integrated to form the complete architecture of the multiplier.

The same implementation has been compared with that on the target device of Virtex4 family.

IV. RESULTS

The following figure shows the functional simulation of the multiplier. It takes two 32-bit floating point numbers and gives their resultant product of 32-bits.

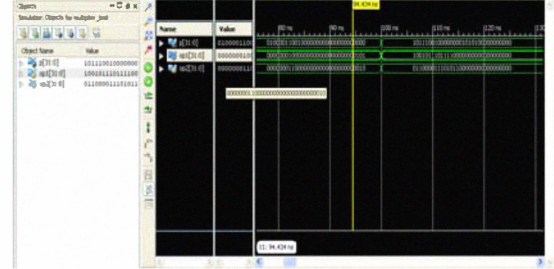


Fig.2. Simulation of Multiplier

The multiplier module was implemented on two families of Xilinx FPGA devices- Spartan 3E and Virtex4. The design and timing information of the multiplier sub-modules have been summarized in the following tables. Comparing the information in these tables we find that the proposed multiplier has a lower delay when implemented on the Virtex4 FPGA.

TABLE 1: SYNTHESIS REPORT OF MODIFIED BOOTH ENCODER

	Spartan3E xc3s500E	Virtex4 xc4vlx15
No. of Slices	68/4656 (1%)	68/6144 (1%)
No. of LUTs	123/9312 (1%)	123/12288(1%)
Delay	11.306 ns	7.244 ns

TABLE 2: SYNTHESIS REPORT OF KOGGE-STONE ADDER

	Spartan3E xc3s500E	Virtex4 xc4vlx15
No. of Slices	204/4656 (4%)	205/6144 (3%)
No. of LUTs	357/9312 (3%)	358/12288(2%)
Delay	18.946 ns	11.097 ns

TABLE 3: SYNTHESIS REPORT OF WALLACE TREE

	Spartan3E xc3s500E	Virtex4 xc4vlx15
No. of Slices	515/4656 (11%)	538/6144 (8%)
No. of LUTs	895/9312 (10%)	935/12288(7%)
Delay	13.708 ns	8.825 ns

TABLE 4: SYNTHESIS REPORT OF MANTISSA MULTIPLIER

	Spartan3E xc3s500E	Virtex4 xc4vlx15
No. of Slices	1307/4656(28%)	1306/6144(21%)
No. of LUTs	2332/9312(25%)	2329/12288(18%)
Delay	28.600 ns	16.316 ns

TABLE 5: SYNTHESIS REPORT OF FINAL MULTIPLIER

	Spartan3E xc3s500E	Virtex4 xc4vlx15
No. of Slices	1269/4656(27%)	1269/6144(20%)
No. of LUTs	2270/9312(24%)	2270/12288(18%)
Delay	34.333 ns	18.783 ns

V. CONCLUSION

We have designed an architecture for a fast floating point multiplier based on the IEEE-754 single precision format. The modules are written in Verilog HDL to optimize implementation on any FPGA. The design is done in such a way that the floating point unit can be effectively interfaced with any processor of 32-bit. The main idea is to increase the speed on the multiplier by reducing delay at every stage using the optimal adder design. We plan to extend this work to design a fast floating point arithmetic logic unit.

VI. REFERENCES

- [1] IEEE standards board, IEEE standard for floating-point arithmetic, 2008
- [2] Paschalakis, S., Lee, P., "Double Precision Floating-Point Arithmetic on FPGAs", *In Proc. 2003, 2nd IEEE International Conference on Field Programmable Technology (FPT '03), Tokyo, Japan, Dec. 15-17, pp. 352-358, 2003*
- [3] Hamacher, Carl, Vranesic, Zvonko, Zaky, Safwat, "Computer Organization" Fifth Edition, pp. 367-390
- [4] Hamid, L.S.A., Shehata, K., El-Ghitani, H., ElSaid, M., "Design of Generic Floating Point Multiplier and Adder/Subtractor Units", *12th International Conference on Computer modelling and Simulation, 2010, pp.615-618*